transforms v3: review and proposal

Round 2 · 2026-11-10 · Baseline: master `7cc2a65` (v2.3.0 plus docs-only commits) · Nothing is implemented. Recommendations, not decisions, except item 10, whose semantics the maintainer settled in discussion.

> **IN ONE PARAGRAPH**
>
> The whole review was repeated from scratch by reviewers who could not see the first round. It reached the same ten changes, added an eleventh, and reversed one recommendation: reading a transform backwards should keep its safety step. The decisions taken in discussion about rejected late samples (item 10) were treated as fixed, and this round worked out exactly how to build and document them. The proposed 3.0 still adds one public type and one error, and removes more than it adds.

1 Background in one minute
2 How the review was done
3 The proposed 3.0 at a glance
4 The eleven changes in detail
5 Considered and left out
6 What can ship in 2.x
7 Release plan
8 Decisions for the maintainer
9 What changed from round 1
10 Limits of this review
11 Prototypes

1 Background in one minute

TERM	MEANING
Frame	A named point of view, such as `map`, `base` or `camera`. A coordinate means something only relative to a frame.
Transform	The recipe for converting coordinates from one frame to another at one instant: a shift (translation) plus a turn (rotation).
Quaternion	Four numbers that encode the turn. It is a pure turn only when its length (norm) is exactly 1. Otherwise it also stretches or shrinks.
Registry	The library's store of transforms over time, arranged as a tree of frames. Each link in the tree is a *hop*.
Lookup	Asking "how do I convert from frame A to frame B at time t?". The registry chains the hops between A and B and interpolates between stored samples.
History window	A registry built with `with_max_age` keeps, per frame, only the samples within `max_age` of that frame's newest timestamp.
Silent wrong answer	A result that looks valid and is not. The repository ranks this as worse than an error or a crash. Every item below is judged against it first.

The repository's scope rule: a capability belongs in the core only if callers cannot build it correctly on the public API, or if the version they would build is a source of wrong answers.

2 How the review was done

> **IN PLAIN TERMS**
>
> Ten reviewers each examined the design from one angle and ran experiments against the real code, without seeing the first round. Ten more tried to disprove their findings. A new draft was then handed to four builders, who made throwaway versions of the changes and ran the existing tests against them, and to six critics, who compared both rounds and ruled on every disagreement.

TECHNICALLY

- **Lens review (20 agents).** Host performance, embedded and scalar, geometry safety, temporal safety, ergonomics, errors, v2 defects, dependents, prior art, migration and release. One reviewer and one skeptic per lens. Reviewers were told not to read `analysis/v3-review/` and were given decisions D1–D6 of item 10 as settled.
- **Stress test (10 agents).** Four prototypes (item 10; items 2–4; items 1 and 5; the deletions) and six critics (scope, alternative designs, dependent cost from real sources, the item 10 contract, consistency against code and round 1, public demand).
- **Models.** Every round 2 agent ran `claude-fable-5-2` at medium effort, verified from the transcripts. Round 1 ran on Sonnet 5.5.
- **Round 1** is kept unchanged under `round1/`.

3 The proposed 3.0 at a glance

`LOUD` means the compiler reports it. `SILENT` means code compiles and behaves differently, so it must be listed in the migration guide. `WIRE` means serialized data is affected.

#	CHANGE	KIND	AGAINST ROUND 1	WHO NOTICES
1	`get_transform_at` returns a cross-time type	`LOUD`	Kept; error rule added	roslibrust_transforms: one return type, one re-export, three test lines
2	Rotations normalized where built	`SILENT` `WIRE`	Kept; bound now proven conditionally	Off-unit rotations read back normalized; results change by up to parts per million
3	`inverse` keeps normalizing; one error variant deleted	`LOUD`	**Changed** (was: conjugate)	Nobody found; no result changes
4	Lookups never return a non-finite value	`SILENT`	Kept	`Ok(inf)` or `Ok(NaN)` becomes `Err`
5	`Disconnected` reported before `NotFoundAt`	`SILENT`	Kept; two error classes	Callers that retry on `NotFoundAt`; no studied user matches a lookup variant
6	Error-shape repairs	`LOUD`	Kept; frame fields reopened	Patterns need `..`; optional rename touches one arm in transforms_io
7	Delete `Point`, `Transformable`, `Localized`, `get_transform_for`	`LOUD` `WIRE`	Kept; cost now stated	No public user found; `Point` payloads stop decoding
8	Delete `Transform::interpolate`	`LOUD`	Kept	No user found
9	Delete `Timestamp::as_seconds`	`LOUD`	Kept	No user found

10	An insert the history window discards returns `SampleExpired`	SILENT	Settled; contract added	Every adapter that sets `max_age`
11	Delete `SameFrameMultiplication`	SILENT	**New**	No user multiplies transforms

Size. One new public type and one new error variant. The deletions alone take the public surface from 104 named items to 87. The prototypes together are roughly +420/−680 library lines.

Left alone. `Stamp` and its serde tags, the flat `RegistryError`, the strict tree and `reparent_frame`, `remove_frame`, no extrapolation, `libm` everywhere, `BTreeMap` history, `latest_common_time`, `Registry::new` and `with_max_age`, the operators, the Non-Goals, MSRV 1.85 and edition 2024. No re-exported type name, module path or constructor name changes.

4 The eleven changes in detail

1 A separate type for cross-time answers

> **IN PLAIN TERMS**
>
> "Where is the object the camera saw half a second ago, relative to where the gripper is now?" That answer spans two instants. Today the library returns it in the same container as an ordinary single-instant transform, labelled with only one of the two times. Nothing stops a program from storing it back as if it were a measurement. This round found a real program path where that happens. The proposal gives the answer its own container that can be read, and cannot be stored or combined.

TECHNICALLY

```
// 2.3: a Transform stamped with target_time only
let tf = registry.get_transform_at("odom", t2, "base", t1, "map")?;
registry.add_transform(tf)?;      // Ok: replaces the honest odom <- base @ t2 sample (x: 1 -> -4)

// 3.0: transforms::geometry::CrossTimeTransform<T>, same method name and argument order
let x = registry.get_transform_at("odom", t2, "base", t1, "map")?;
x.target_frame();  x.target_time();
x.source_frame();  x.source_time();
x.translation();   x.rotation();  // Vector3, Quaternion
registry.add_transform(x);        // does not compile
```

- **Evidence.** roslibrust_transforms wraps this method and also inserts then publishes, so the overwritten sample goes out on `/tf`. The re-insert succeeds only when the target is the source's pinned parent.
- **What it closes.** The accident. Not the capability: one `Transform::new` expression rebuilds an insertable transform from the accessors. Today that takes zero lines.
- **Why not delete the method.** The outside replacement restamps one leg through a public constructor, which defeats the equal-stamp guard. That is a wrong-answer generator, so the scope rule keeps it in the core.
- **Error rule (new).** Both legs are resolved. The reported error is chosen by rank `UnknownFrame` > `Disconnected` > other, target leg first. `NotFoundAt` names the caller's target and source; `Disconnected` names (endpoint, fixed frame). Without this, the error variant still depends on `source_time`.
- **Shape.** Private fields, six accessors, no `Mul`, `inverse`, serde or conversion. `Clone` and `PartialEq` are optional.
- **Measured.** 43,427 calls: all 2,060 `Ok` results are bit-identical to 2.3.0. roslibrust type-checks after changing one return type.

2 Rotations are made unit where they are built

IN PLAIN TERMS

A rotation is stored as four numbers whose combined length should be exactly 1. If the length is slightly more than 1, the rotation also enlarges whatever it turns. The library accepts rotations up to one millionth off and stores them as given, so each link in a chain stretches the result a little. The proposal rescales every rotation to length 1 at the moment a transform is created. Rotations that are already at length 1, to within rounding, are left untouched, so doing it twice changes nothing.

TECHNICALLY

```
// Called by Transform::new, static_between and Deserialize, after validate().
// Plain arithmetic: no `?`, no fallback arm. Bit-identical to Quaternion::normalize.
let n2 = q.w * q.w + q.x * q.x + q.y * q.y + q.z * q.z;
if (n2 - 1.0).abs() > 8.0 * f64::EPSILON {
    let k = 1.0 / libm::sqrt(n2);
    q = Quaternion::from_wxyz(q.w * k, q.x * k, q.y * k, q.z * k);
}
```

MEASURED EFFECT OF ACCEPTED OFF-UNIT ROTATIONS (2.3.0)	ERROR
Four hops of 10 km at `w = 1 + 0.9e-6`	0.108 m
Ten hops of 10 m at norm 1 ± 9e-7, at 100 m	0.81 mm
Same frame, stored instant against midway between samples (`w = 1 − 9e-7`)	norm −9.0e-7 against −4.5e-7
Unit quaternion rounded to `f32` , worst norm error	5.0e-8

- **Reach.** The rule changes 200,000 of 200,000 `f32` -sourced rotations and 0 of 200,000 `f64` -normalized ones. dimos already trips on the effect: a three-hop lookup over rotations its own ingest admits fails its own validity re-check.
- **Why the band.** Normalizing is not idempotent: a second pass changes the bits of 38% of once-normalized quaternions. After one normalization the squared norm is within 7 EPS of 1, proven if `libm::sqrt` is correctly rounded (tested on 6e9 values, not proven), and within 8 EPS if it is only faithful. Observed maximum 4 EPS over 3.4e9 samples; rebuilding a built value changed 0 bits.
- **Declared changes.** Off-band inputs: `tf.rotation()` no longer equals the quaternion passed in, and results change by up to 2e-6 relative per hop. In-band inputs: 0 bits changed in 100,000 lookups. A flattened product of two band-edge transforms is now accepted on insert.
- **Open.** Normalizing only at `Buffer::insert` is smaller (one test flips, not three) and leaves built, never stored transforms off-unit. The constructor site is recommended.
- **Residual.** Insert validates and does not normalize, so a re-published derived rotation within 1e-6 is stored as given.

3 `inverse` keeps its normalization

IN PLAIN TERMS

Reading a transform backwards requires undoing its rotation. For a rotation of length exactly 1 that means flipping three signs. Today the library first rescales the rotation as a precaution. Round 1 proposed dropping the precaution to save a little time. Round 2 built both versions: dropping it fixes nothing, changes the last digits of about half of all answers, and makes one existing test fail about one run in ten. The recommendation is now to keep the precaution and only tidy up how its failure is reported.

TECHNICALLY

	MASTER	(A) CONJUGATE	(B) KEEP NORMALIZING
Lookup results with changed bits, unit inputs	n/a	46.5%	0
`registry_chain_roundtrip_composes_to_identity` fails	1 in 3e6 cases	≈ 10.6% of runs	as master
`inverse()` of a rotation collapsed by `*`	`Err`	`Ok`	`Err`
One-hop inverting lookup, host	78.1 ns	61.1 ns	79.6 ns

- **Recommendation (b).** Keep the normalization, written without `?`. Report a non-finite norm as `NonFiniteValues` and a near-zero norm as `NonUnitRotation(norm)`. Delete `TransformError::QuaternionError`.
- **The variant is reachable today** through public `*` then `inverse()` (72 multiplications from `w = 1 + 1e-6`), so it cannot simply be deleted as dead.
- **(a) is numerically sound.** Its error is the same order as the forward error the crate already has; no Newton step is needed. It stays available later as a measured performance change.
- **Dissent.** One critic ranks (b) lowest because it needs a rule for a zero norm.

4 No lookup returns a non-finite value

> IN PLAIN TERMS
>
> With extreme numbers, arithmetic overflows to "infinity" or to "not a number". Today some lookups return such an answer and mark it as success. A controller that receives "infinity metres, OK" is worse off than one that receives an error. The proposal checks the answer once, just before it is returned.

TECHNICALLY

```
// 2.3
registry.get_transform("left", "right", t)   // Ok(Transform { translation: (-inf, inf, NaN), .. })
// 3.0: one check at the exit of get_transform and get_transform_at
registry.get_transform("left", "right", t)   // Err(RegistryError::NonFiniteValues)
```

- Reproduced on master for ancestor-ward, sibling, interpolated, rotated and static chains, three `get_transform_at` cases, and a chain whose true result is finite (5e307).
- +19 lines; the cost is not distinguishable from zero on host. It changes no number.
- **Not on `*`.** Round 1's prototype also gated `*`; that gate fails an existing property test and closes no lookup hazard.
- It changes a documented behaviour, so it is listed for 3.0. A maintainer ruling could move it to 2.x.

5 `Disconnected` is reported before `NotFoundAt`

> IN PLAIN TERMS
>
> If two frames are not linked at all, asking for the transform between them should say so. Today, when the data also happens to miss the requested time, the library reports a timing problem instead. The same question then gets a different kind of answer depending on the time asked, although the real cause is that the frames are not connected.

TECHNICALLY

- Precedence becomes `UnknownFrame` > `Disconnected` > `NotFoundAt`, as `latest_common_time` already decides. The change deletes one fallback and its plumbing: +41/−66 lines; one test flips.

- **Two error classes change.** In 144,000 queries over two clocks, 30,294 differ, all for disconnected pairs: `NotFoundAt` to `Disconnected`, and with a custom clock `TransformError(TimestampError(..))` to `Disconnected`. No other lookup and no `latest_common_time` result changes.
- **Correction.** Round 1 wrote that waiting "never will" help a disconnected pair. A later insert can connect two trees. The reason for the change is that the variant must not depend on the time.
- `get_transform_at` needs the cross-leg rule of item 1 to get the same property.
- The resolver rewrite is internal and is not a release item. Both rounds measured it slower on long root-ward chains.

6 Error shapes

> **IN PLAIN TERMS**
>
> Small repairs to how errors are laid out, so that they can gain detail later without breaking programs, and so that one error stops printing two different times as the same number. One question is open: should an error about a rejected insert name the frame it concerns?

TECHNICALLY

- `TimestampMismatch` **becomes a unit variant.** Its `f64` seconds payload prints equal values for stamps 1 ns apart. `TransformError<T>` stays rejected: it breaks roslibrust's public conversion trait.
- `#[non_exhaustive]` **on struct variants.** No dependent breaks.
- **Open: a child** `frame` **field on the four structural insert errors**, printed by `Display`. For: after item 10 one insert error names its frame and four do not; the failing member of a batch cannot be found outside without giving up atomicity; unit to struct needs a major. Against: it names an edge, not a member; the only batch caller is an unmerged PR. Critics split two for, one against, one undecidable. Leaning yes.
- **Optional:** rename `ReparentingNotSupported` to `ParentMismatch`.

7 Delete `Point` and its two traits

> **IN PLAIN TERMS**
>
> The library ships a small helper that applies a transform to a position and checks that the frames and times match. No public project uses it; they all convert results into their own maths library. Since it can be rebuilt outside in a dozen lines, the scope rule says it does not belong in the core. The price is that people who apply transforms by hand lose those checks, and that the first example in the README must be rewritten.

TECHNICALLY

- **For.** Zero call sites in seven public code bases, by three search methods. `Point::transform` reads only public accessors and validates no numbers. A 12-line outside function gave the same errors.
- **Against.** The only two outsiders who posted code in the tracker used `Point` (#43, #65). Issue #43 shows its frame check catching a wrong-direction application. Private users are invisible.
- **Declared cost.** For hand application, seven slips become silent for a position (swapped lookup, other instant, other frame, applied twice, three hand-math slips). A pose written as a `Transform` keeps the checks through `*`.
- **Conditions.** Move the application formulas to `Transform`'s rustdoc; re-point the SciPy golden vectors without touching their digits; rewrite the README headline; give 2.x users a removal notice (`#[deprecated]`, or changelog and README).
- **Not taken.** Keeping `Point` and `Transformable` (−79 lines instead of −369): `Point` would need hardening, and it cannot apply a cross-time result correctly. Three critics rule for deletion; the deletions prototype and the demand critic are more cautious.

8 Delete `Transform::interpolate` 9 Delete `Timestamp::as_seconds`

> **IN PLAIN TERMS**
>
> Two public functions that nothing uses. The first duplicates what the registry does internally. The second fails for every real clock time.

TECHNICALLY

- **8.** No registry path calls it, no user found; −127 lines and two error variants. The test that used it as a reference gets a 12-line in-test reference instead. Today's reference is blind: with the interpolation weights swapped inside the crate, the existing test still passes.
- **9.** `Err` above 2^53 ns, about 104 days. `as_seconds_lossy` stays. `TimeError::AccuracyLoss` stays for custom clocks and needs its doc reworded.

10 An insert the history window discards is an error

> **IN PLAIN TERMS**
>
> A registry with a history window keeps only recent samples. If one sample arrives with a timestamp far in the future, the window jumps ahead with it. From then on every normal sample counts as "too old": it is thrown away, and the library still answers "OK". The robot's position silently freezes. The decision taken in discussion: the library must say so. Such an insert returns an error and stores nothing.
>
> The library cannot know which timestamp is the wrong one, because it never looks at a clock. So the error is a report, not a diagnosis, and the future-stamped sample itself is still accepted.

SETTLED IN DISCUSSION (2026-11-10)

	DECISION
D1	An insert strictly older than the frame's newest stamp minus `max_age` returns `RegistryError::SampleExpired { frame, stamp, newest }` and stores nothing. A sample exactly on the boundary is accepted, as in eviction.
D2	`add_transforms` fails as a whole and stays equal to inserting one by one. Each member is measured against its frame's running newest stamp, earlier members included.
D3	No `max_age` : never reported. Statics unaffected. Where the subtraction is not representable, the sample is accepted. The `reparent_frame` seed is exempt.
D4	Measuring against the batch's newest stamp (independent of order) is rejected: the batch would stop equalling the loop, which the docs promise and a property test pins.
D5	No `clear_history` .
D6	2.x keeps the silent drop. The error is not clock-reset detection.

```
// 10 s window, one frame; on Err nothing is stored
registry.add_transforms([at(50), at(100)])   // Ok; at(50) is evicted by at(100), as in the loop
registry.add_transforms([at(100), at(50)])   // Err(SampleExpired { stamp: 50, newest: 100 })
```

WHAT ROUND 2 ADDS

- **The trap.** All ten reviewers found it independently: testing each member against the stored buffer breaks D2 on frames the registry already holds (`[at(100), at(50)]` stores 100, then fails). The check phase needs a per-frame running newest stamp. The existing property test notices in 2 of 40 runs; a biased one in 40 of 40.
- **Round 1's prototype did not have the bug.** It differs in adding `clear_history` and in describing `newest` as stored.
- **Order.** Expiry is the last insert check. An expired sample with the wrong parent reports the parent error. So a sample that reports `SampleExpired` has the frame's parent and kind.

- **Placement.** A separate `Buffer::check_expiry` called last by the single insert and by the batch check phase; `Buffer::check` is unchanged. +118/−6 lines, three existing tests flip.
- **One threshold.** Reuse eviction's `checked_sub`. A `duration_since` form silently stops rejecting for some custom clocks.
- **Payload.** From a batch, `stamp` is the first failing member, not necessarily the culprit, and `newest` can be an earlier member that the registry does not hold after the error.

RECOVERY, BY CAUSE

Recover only when `newest` is judged to be the wrong stamp. Applied mechanically, one late sample wipes a healthy frame. The error does not return the transform.

CAUSE	ROUTE	NOTES
One frame's newest stamp is wrong	`remove_frame`, then re-add a sample the caller kept	Works for every time type and stamp. Releases the parent and kind pins until the re-add. The subtree below stays intact.
The clock stepped back	`remove_transforms_before(c)`, `c` strictly later than every dynamic sample	Drains every dynamic frame, keeps statics and pins. `c` equal to the newest stamp keeps the sample. No such `c` at the type's maximum.
Everything	A new `Registry`, or `remove_frame` over tracked names	The registry cannot enumerate frames. A new registry also drops statics.

Round 1 also named two `reparent_frame` calls. That stays out of the docs: it needs a fabricated transform and its second seed is unchecked.

CONSEQUENCES TO DOCUMENT

- A zero window rejects every out-of-order sample.
- After a backward jump larger than `max_age`, errors stop once the stream reaches newest minus `max_age`; lookups then blend the two epochs (1000 s to 100 s, 10 s window: 890 errors, then a blended lookup at 990.5 s).
- A sample stamped ahead by less than `max_age` is never reported, and lookups interpolate toward it (up to 3.47 m off in the probe, all `Ok`).
- transforms_io and dimos expose no recovery call; roslibrust only `remove_frame`. Two adapters log each rejection while holding the write lock. One expired member rejects a whole TF message in dimos.
- tf2 uses the same boundary (`TF_OLD_DATA`). Its users ask for a whole-buffer reset and fewer warnings, never for the late sample to be accepted.

Draft rustdoc, the MIGRATION row and the AGENTS.md bullet are in `sample-expired-contract.md`.

11 Delete `SameFrameMultiplication`

> #### IN PLAIN TERMS
>
> Chaining two transforms is allowed when the first ends where the second begins. The library has one extra rule on top that rejects a harmless case: chaining with the "no movement" transform the registry itself returns for a frame relative to itself, but only when it stands on the right. Removing the rule makes the code match the documented rule. It is safe only once item 1 is in place.

TECHNICALLY

```
get(a, a) * get(a, c)   // Ok today
get(a, c) * get(c, c)   // Err(SameFrameMultiplication) today; Ok after
```

- **Only with item 1.** Without it, `world_camera @ t2 * get_transform_at(camera, t2, camera, t1, world)` would become `Ok` with the t1 pose stamped t2. The same product already composes on the left today.

- Of 324 frame pairings, 64 change from `Err` to `Ok` and 52 change variant to `IncompatibleFrames`. No registry lookup changes (72,000-line trace identical). Four tests flip.
- `Transform::new` gains no `parent == child` rule: the registry's identity is rebuilt through it.

5 Considered and left out

ITEM	REASON
Conjugate `inverse`	Item 3. Sound, closes nothing, changes bits
Public `Rotation` or `Isometry`; translation bound; Newton step	Items 2–4 close the same hazards without a new type or constant. The Newton step changes the bits of 45% of two-hop products
A maximum interpolation gap	**Open, not rejected.** Callers cannot build it, and it would close the in-window future stamp of item 10. It is additive and has no settled design; a registry-wide bound is the wrong shape for mixed-rate edges (a bound that serves a 1 Hz edge still passes 0.245 m on a 100 Hz edge)
Required `max_age`	Every studied user already sets it. Round 1's reason was wrong: it does bound the gap
`clear_history`	Settled (D5)
Operators to named methods	The slip is always a compile error. An additive diagnostic attribute fixes the message in 2.x
Deleting `get_transform_at`; a slice `apply`; `Resolved`; `TransformError<T>`	Rebuilt wrongly outside, buildable outside, closes nothing, breaks a dependent
Deleting `TimePoint::checked_sub`	Refuted: eviction silently stops for a custom clock
Optional approx; `BTreeMap` for frames; signed `Timestamp`; `f32`; `FrameId`	Deferred or additive
"3.0 changes no byte and no number"	Rejected as a rule. Adopted as "no number that is right today changes", which the conjugate fails

6 What can ship in 2.x

- The 2.4 proposal: empty frame names and the documentation corrections. Reword its Part C item 5, whose recovery advice has the same holes as round 1's.
- The removal notice for the Point family, and the `TimePoint` diagnostic attribute.
- Docs: `Display` of time-carrying errors is lossy; cross-time errors may name the fixed frame; the `latest_common_time` example panics on an all-static chain; `f64` runs in software on every gate target by default.
- Two doc recipes adapters get wrong today: a "latest" lookup, and a checked ROS time conversion.
- Tests: a frozen-bytes case with a non-identity rotation. Today a reordered quaternion is not caught.
- Internal: delete the redundant `latest_timestamp` field, `Buffer.child` and `InsertError::ChildFrameMismatch`.

7 Release plan

- **Order.** 2.4, then the 3.0 prototype on a branch through the full gate, then one `3.0.0-rc` built against transforms_io and roslibrust_transforms, then the release.
- **Warrant.** Item 10 cannot ship in a minor, so a 3.0 is a given. It is not urgent.
- **Acceptance.** "3.0 changes no number that is right today", with a listed set of allowed differences: item 2 for off-band inputs, item 4, item 5, cross-time error payloads; for inserts `Ok` to `SampleExpired` and `Err(NonUnitRotation)` to `Ok` for products of band inputs.
- **Wire.** Layouts stay frozen; 3.0 reads every 2.x `Transform` payload. Off-band rotations are normalized on decode; `Point` payloads stop decoding.
- **Tests.** Four of nine `v2_compatibility` tests flip. Nine other pinned tests flip; 17 are deleted with their subjects.
- **AGENTS.md.** Ten of sixteen invariant bullets change text. Non-negotiables and Non-Goals are untouched.
- **MIGRATION.** One row per item; per-adapter recovery recipes for item 10.

8 Decisions for the maintainer

#	QUESTION	RECOMMENDATION
1	`inverse`: conjugate, keep normalizing and delete the variant, or status quo?	Keep normalizing. Changed from round 1
2	Normalize at the constructors and `Deserialize`, or only at insert?	Constructors and `Deserialize`
3	Point family: delete all four, or keep `Point` and `Transformable`?	Delete all four, with the stated cost accepted
4	2.x notice: `#[deprecated]`, or changelog and README?	Changelog and README
5	Child frame field on structural insert errors?	Leaning yes; the door closes for a major cycle
6	Six accessors on the cross-time type, or two?	Six
7	Finiteness check: 3.0 or a 2.x fix?	3.0, unless ruled a bug fix
8	Delete `SameFrameMultiplication`?	Yes, only with item 1
9	Rename `ReparentingNotSupported`?	Optional
10	Dev-dependency on 2.3 for a differential harness?	Maintainer's call
11	Round 1's policy sentence on what a minor may change	Still open
12	Item 10	Decided (D1–D6)

9 What changed from round 1

TOPIC	ROUND 1	ROUND 2
`inverse`	Conjugate; 28% of lookups change bits; 10–13% faster	Keep normalizing. 46.5% change bits under the conjugate; one test becomes flaky; one `Err` becomes `Ok`
`QuaternionError`	Unreachable after item 2	Reachable today through public `*`
Dead-band bound	7 EPS, unproven	Proven given a correctly rounded `sqrt`

Second normalization changes bits	15.6%	38%; 15.6% is the third pass
Item 5	One error class; waiting "never will" help	Two classes; a later insert can connect trees; cross-leg rule needed
Cross-time errors	Open	Rule given and prototyped
Item 1 evidence	A pinned test	A live overwrite-and-broadcast path
Point family	Delete; cost not stated	Delete; seven silent slips stated
`SameFrameMultiplication`	No position	Delete, with item 1
Frame fields on insert errors	Not in 3.0	Open, leaning yes
Item 10 recovery	Drain "past the newest sample"; two reparents	By cause; `remove_frame` first; strict cutoff and its limits; guard against mechanical use
Item 10 `newest`	"Newest stored instant"	Can be an unstored batch member
Item 10 cost	41 against 74 ns	Not reproduced: 63.6 against 64.3 ns
Required `max_age`	"Bounds memory only"	Wrong; it bounds the gap. Still rejected
Maximum gap	Rejected	Open, additive, design first
`v2_compatibility`	Three flip, six stay	Four flip, five stay
AGENTS.md bullets	Six	Ten of sixteen

Round 1 was right against round 2's own draft on three points: `f64` runs in software on every gate target by default, the condition on the resolver rewrite, and the remedy for the `interpolate` test reference. Round 1's item 10 prototype was correct on atomicity and check order.

10 Limits of this review

- The full gate (`tests/test_all.sh`) ran on no prototype. Prototypes ran `cargo test` in both feature modes, most with `serde` , plus clippy and fmt on the library.
- Nothing was measured on hardware. Embedded figures come from QEMU. Host timings were taken on a shared machine; differences under about 3 ns are noise.
- Only roslibrust_transforms was type-checked against a prototype. Other downstream counts are from reading sources.
- `libm::sqrt` correct rounding is tested, not proven.
- The 10.6% flake rate comes from a replica of the test's generators.
- Private users are invisible. "Zero users" means zero in seven public code bases.
- The frame-field question was not settled by evidence.

11 Prototypes

Library-only diffs against `7cc2a65` in `prototypes/` ; each applies cleanly. They are evidence, not patches.

FILE	CONTENT	LINES	EXISTING TESTS THAT FAIL
`sample-expired.diff`	Item 10	+118/−6	3, pins of the old contract

`sample_expired_tests.rs`	15 new tests for item 10	445	n/a
`normalize-finite-check-inverse-b.diff`	Items 2, 4, 3 as recommended	+59/−11	4, all intended
`normalize-finite-check-inverse-a-conjugate.diff`	Items 2, 4 with the conjugate	+56/−19	4, plus one flaky property
`cross-time-type-and-precedence.diff`	Items 1 and 5	+214/−118	1; 12 lines stop compiling
`precedence-only.diff`	Item 5 alone	+41/−66	1
`deletions.diff`	Items 7, 8, 9, 11	+29/−546	4 flip (item 11)